
APACHE AIRFLOW 3.1.7



STAGE 4 / PRODUCTION

Production

Celery + Redis, Monitoring, Scaling & SLA

Target:	Backend / Distributed Systems Engineer
Duration:	Ngày 10-13 (2-3 giờ/ngày)
Focus:	Production deployment
Version:	Apache Airflow 3.1.7

Mục lục

1. Tổng quan production deployment

2. Docker Compose — CeleryExecutor + Redis

2.1. File docker-compose.prod.yml

2.2. Các service thêm vào

3. Redis Configuration cho Production

4. Prometheus + Grafana Monitoring

4.1. Metrics endpoint

4.2. Key metrics cần monitor

5. Scaling Patterns

5.1. Vertical Scaling

5.2. Horizontal Scaling

5.3. Database Connection Pooling

6. SLA & Alerting

7. Bài tập thực hành

1. Tổng quan production deployment

Khi chuyển từ development sang production, có 3 thay đổi lớn cần thực hiện:

- **Executor:** LocalExecutor → CeleryExecutor (hoặc KubernetesExecutor)
- **Monitoring:** Thêm Prometheus + Grafana để theo dõi hiệu năng
- **SLA & Alerting:** Đặt ngưỡng thời gian, cấu hình cảnh báo

Stage 4 hướng dẫn cách triển khai Airflow với CeleryExecutor và Redis làm message broker — đây là stack phổ biến, ổn định, và phù hợp với hệ thống đã có Redis/Kafka.

2. Docker Compose — CeleryExecutor + Redis

File `docker-compose.prod.yml` mở rộng từ Stage 1, thêm Redis, Celery Worker, Triggerer, và Flower (monitoring UI cho Celery).

2.1. File docker-compose.prod.yml

```

# docker-compose.prod.yml (CeleryExecutor + Redis + Flower)
version: '3.8'
x-airflow-common:
  &airflow-common
  image: apache/airflow:3.1.7
  environment:
    &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__BROKER_URL: redis://:@redis:6379/0
    AIRFLOW__CELERY__RESULT_BACKEND: db+postgresql://airflow:airflow@postgres/airflow
    # Redis XCom Backend – thay vì S3
    AIRFLOW__CORE__XCOM_BACKEND: airflow.providers.redis.xcom_backends.RedisXComBackend
    AIRFLOW__REDIS__HOST: redis
    AIRFLOW__REDIS__PORT: 6379
    AIRFLOW__REDIS__DB: 1
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL: 120
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins

services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - ./redis.conf:/usr/local/etc/redis/redis.conf
    command: redis-server /usr/local/etc/redis/redis.conf
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5

  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data

  airflow-webserver:
    <<: *airflow-common
    command: webserver

```

```

- "8080:8080"

airflow-scheduler:
  <<: *airflow-common
  command: scheduler

airflow-worker:
  <<: *airflow-common
  command: celery worker
  # Scale: docker compose up --scale airflow-worker=4

airflow-triggerer:
  <<: *airflow-common
  command: triggerer

flower:
  <<: *airflow-common
  command: celery flower
  ports:
    - "5555:5555"
  environment:
    <<: *airflow-common-env

volumes:
  postgres-db-volume:

```

2.2. Các service thêm vào

Bảng 1: Service mới trong production deployment

Service	Vai trò	Port
redis	Message broker cho Celery + XCom backend (DB 0 + DB 1)	6379
airflow-worker	Celery worker — thực thi task được gửi từ scheduler	Không expose
airflow-triggerer	Chạy deferrable operators/sensors async	Không expose
flower	Web UI monitoring cho Celery — xem queue, worker, task status	5555

Scale worker

Docker Compose hỗ trợ scale worker động: `docker compose up --scale airflow-worker=4 -d`. Điều này tạo ra 4 worker container, mỗi worker có thể xử lý task độc lập.

3. Redis Configuration cho Production

Redis làm 2 vai trò: (1) message broker cho Celery (DB 0), (2) XCom backend (DB 1). Cấu hình dưới đây tối ưu cho cả 2:

```
# redis.conf (append-only persistence)
# DB 0: Celery broker – tasks ephemeral, không cần persistence nghiêm ngặt
# DB 1: XCom backend – cần lưu tạm thời, có TTL

appendonly yes
appendfsync everysec
maxmemory 256mb
maxmemory-policy allkeys-lru

# Cấu hình database count (mặc định 16, đủ cho phân chia)
databases 4
```

Bảng 2: Giải thích cấu hình Redis

Tham số	Ý nghĩa	Giá trị
<code>appendonly</code>	Enable AOF persistence	<code>yes</code>
<code>appendfsync</code>	Tần suất fsync AOF	<code>everysec</code> (cân bằng hiệu năng/độ bền)
<code>maxmemory</code>	Giới hạn bộ nhớ	<code>256mb</code> (tùy mức độ sử dụng)
<code>maxmemory-policy</code>	Chính sách xóa khi hết bộ nhớ	<code>allkeys-lru</code> (xóa key ít dùng nhất)

4. Prometheus + Grafana Monitoring

Airflow 3.x hỗ trợ metrics thông qua StatsD hoặc Prometheus native exporter. Cài đặt Prometheus để scrape metrics, Grafana để visualize.

4.1. Metrics endpoint

```
# airflow.cfg
[metrics]
statsd_on = False
# Airflow 3.x hỗ trợ Prometheus native exporter hoặc StatsD → Prometheus
```

4.2. Key metrics cần monitor

Bảng 3: Các metrics quan trọng trong Airflow production

Metric	Ngưỡng cảnh báo	Ý nghĩa
<code>airflow_scheduler_heartbeat</code>	> 0	Scheduler đang hoạt động
<code>airflow_dag_processing_total_parse_time</code>	< 30s	Thời gian parse DAG không quá chậm
<code>airflow_executor_queued_tasks</code>	< 100	Queue không bị tắc
<code>airflow_executor_running_tasks</code>	< max worker slots	Worker không quá tải
<code>airflow_task_failures</code>	Spike detection	Task failure rate đột biến
<code>airflow_dagrun_duration</code>	SLA-based	DAG chạy quá lâu

Cảnh báo quan trọng nhất

Scheduler heartbeat là metric quan trọng nhất. Nếu heartbeat dừng, toàn bộ hệ thống ngừng hoạt động. Alert phải có độ nhạy cao nhất cho metric này.

5. Scaling Patterns

5.1. Vertical Scaling

Vertical scaling tăng cường phần cứng cho 1 node — phù hợp cho LocalExecutor:


```
AIRFLOW__CORE__PARALLELISM = 32
AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG = 16
AIRFLOW__CORE__DAG_CONCURRENCY = 16
```

5.2. Horizontal Scaling

Horizontal scaling thêm node — phù hợp cho CeleryExecutor và KubernetesExecutor:

```
# Scale workers trong Docker Compose
docker compose up --scale airflow-worker=8 -d

# Auto-scaling với K8s HPA (KubernetesExecutor)
kubectl autoscale deployment airflow-worker \
  --min=2 --max=10 --cpu-percent=70
```

5.3. Database Connection Pooling

Khi có nhiều worker kết nối đến DB, cần cấu hình connection pool để tránh quá tải:

```
AIRFLOW__DATABASE__SQL_ALCHEMY_POOL_SIZE = 10
AIRFLOW__DATABASE__SQL_ALCHEMY_MAX_OVERFLOW = 20
AIRFLOW__DATABASE__SQL_ALCHEMY_POOL_RECYCLE = 3600
```

Bảng 4: Tham số connection pool

Tham số	Ý nghĩa	Khuyến nghị
<code>POOL_SIZE</code>	Số kết nối tối thiểu duy trì	10 cho production
<code>MAX_OVERFLOW</code>	Số kết nối vượt trên pool_size khi cần	20 (gấp 2 lần pool_size)
<code>POOL_RECYCLE</code>	Thời gian recycle kết nối (giây)	3600 (1 giờ)

6. SLA & Alerting

SLA (Service Level Agreement) trong Airflow là thời gian tối đa cho phép để 1 task hoặc DAG hoàn thành. Nếu vượt quá, hệ thống sẽ gửi cảnh báo.

```

from datetime import timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def calculate_tax(**context):
    pass

with DAG(
    'production_invoice_pipeline',
    schedule=timedelta(hours=1),
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['production', 'invoice'],
    default_args={
        'sla': timedelta(minutes=45), # SLA cho từng task
        'email': ['ops@company.com'],
        'email_on_sla_miss': True,
        'email_on_failure': True,
        'email_on_retry': False,
    }
) as dag:

    # Task với SLA riêng
    critical_task = PythonOperator(
        task_id='tax_calculation',
        python_callable=calculate_tax,
        sla=timedelta(minutes=15), # Task này phải xong trong 15 phút
    )

```

Bảng 5: Các tham số alerting trong Airflow

Tham số	Tác dụng	Khuyến nghị
<code>sla</code>	Thời gian tối đa task phải hoàn thành	Tính từ <code>execution_date</code>
<code>email_on_failure</code>	Gửi email khi task thất bại	True cho production
<code>email_on_sla_miss</code>	Gửi email khi vượt SLA	True cho task quan trọng
<code>email_on_retry</code>	Gửi email khi task retry	False (để tránh spam)

7. Bài tập thực hành

Bảng 6: Bài tập Stage 4

STT	Bài tập	Kỹ năng
1	Chuyển LocalExecutor → CeleryExecutor + Redis	Production setup
2	Scale worker: <code>docker compose up --scale airflow-worker=3</code>	Horizontal scaling
3	Truy cập Flower UI (localhost:5555) và observe task queue	Monitor Celery
4	Setup Prometheus scraping endpoint, tạo 1 Grafana panel	Monitoring stack
5	Đặt SLA tên task và simulate miss → check email/alert	SLA configuration
6	Simulate worker crash: <code>docker kill <worker></code> → observe retry	Fault tolerance
7	Test DB connection pool: monitor <code>pg_stat_activity</code>	DB optimization

Lưu ý về bảo mật

Trong production, hãy: (1) Đổi mật khẩu mặc định admin, (2) Enable Fernet key để mã hóa connections, (3) Dùng Vault hoặc AWS Secrets Manager cho credentials, (4) Giới hạn truy cập Redis bằng password hoặc ACL, (5) Enable HTTPS cho webserver.